

UNIVERSIDAD ANDINA DEL CUSCO
FACULTAD DE INGENIERÍA Y ARQUITECTURA
ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS



ACTIVITY : IF

COURSE : Realidad Virtual

PROFESSOR : Zuñiga Rojas Gabriela

SEMESTER : VIII

STUDENTS :

- Carpio Letona, Rafit Holger
- De los Rios Cano Franco Sebastian
- Paredes Claros, Adrian Mauricio

2026

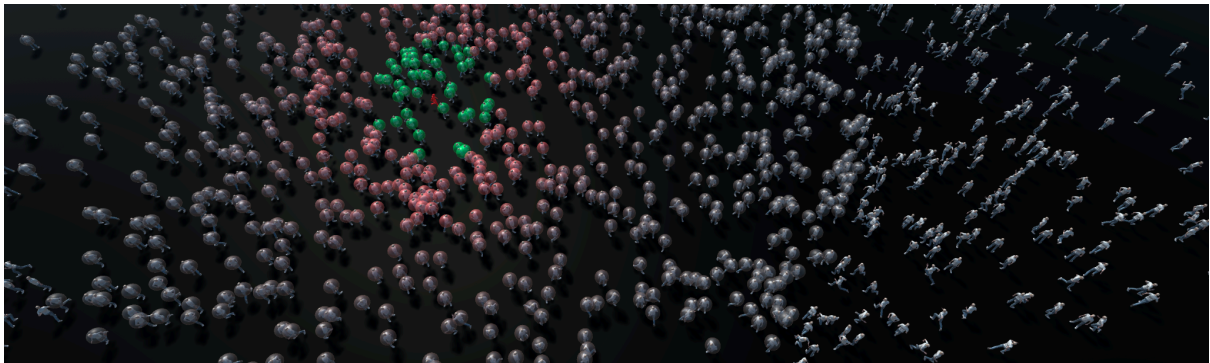
Cusco - Perú

Foveated Animations for Efficient Crowd Simulation

FLORIN-VLADIMIR STANCU, University of Leeds, United Kingdom

TOMER WEISS, New Jersey Institute of Technology, USA

RAFAEL KUFFNER DOS ANJOS, University of Leeds, United Kingdom



Crowd simulation where the focus point is depicted in red.

Introduction to Virtual Reality (VR)

Virtual Reality applications using Head-Mounted Displays (HMDs) require high-performance rendering pipelines due to high pixel densities, wide fields of view, and low latency tolerance. When VR applications involve virtual crowds (hundreds or thousands of interactive, animated characters), calculating bone transformations, IK solvers, and animation blending per frame creates severe CPU and GPU performance bottleneck.

Central Vision vs. Peripheral Vision

The Human Visual System (HVS) processes visual information differently across retinal locations.

- **Central (Foveal) Vision:** Concentrated within the central $\approx 5^\circ$ of the visual field (the fovea). It features a high density of cone photoreceptors, providing high spatial acuity and detail perception. Any degradation in quality or frame rate in this region is instantly noticeable.
- **Peripheral Vision:** Characterized by lower cell density and larger "pooling regions" in the visual cortex. Peripheral vision perceives groups of elements statistically through visual pooling and is susceptible to the crowding effect (difficulty distinguishing individual items surrounded by clutter). However, peripheral vision remains sensitive to temporal changes and flicker, peaking around $20^\circ - 50^\circ$ eccentricity.

The Research Problem

While conventional foveated rendering reduces spatial pixel shading or geometry resolution in non-focused regions, it does not alleviate animation computation costs.

- **Core Question:** *Can gaze-tracking data and peripheral perception properties be leveraged to reduce or completely pause skeletal animation updates in peripheral regions without the user noticing a degradation in quality?*
- **Tested Approaches:**
 - a. **Dynamic Foveation:** Decreasing animation update frequencies quadratically as distance from the focal point increases.
 - b. **Full-Stop Foveation:** Completely halting skeletal animation for all agents outside the foveal vision area.

Key Findings

- **Massive Efficiency Gains:** Full-Stop foveation achieved up to a 99.3% reduction in required animation operations, while Dynamic foveation yielded a 78.7% reduction.
- **Perceptual Superiority of Full-Stop:** Counterintuitively, completely pausing limb animations in peripheral vision is less noticeable to users than updating them at reduced frame rates (e.g., 5–10 Hz). Low-frequency updates tend to cross the Critical Flicker Frequency (CFF) threshold and trigger flicker awareness.
- **Visual Crowding Impact:** Dense, erratic crowd movements mask omissions in individual limb animation much better than organized or sparse formations.

State of the Art: Foveated Animation as an Optimization Solution

Traditional foveated rendering techniques (gaze-contingent rendering) have gained widespread adoption in computer graphics and immersive virtual reality environments owing to their ability to optimize spatial rendering workloads. However, conventional implementations focus exclusively on spatial degradation—such as lowering pixel shading rates, altering Variable Rate Shading (VRS) masks, or reducing peripheral mesh geometry. Despite these advancements, standard spatial techniques completely ignore temporal animation evaluation costs, failing to alleviate the heavy computational bottleneck caused by real-time skeletal transformations, inverse kinematics (IK) solvers, and skinning operations required when rendering large-scale interactive crowds.

To overcome these limitations, the concept of Foveated Animation was introduced as an integrated spatial-temporal solution. By aligning character skeletal evaluation frequencies directly with the user's real-time gaze coordinates captured via high-frequency eye-tracking hardware, the system leverages the physiological constraints of the Human Visual System (HVS). Specifically, central (foveal) vision covers a cone of approximately 5° of the visual

field, demanding 100% animation fidelity (R_t). In contrast, peripheral vision processes motion through statistical pooling and exhibits visual crowding, allowing peripheral animation frequencies to be aggressively suppressed without compromising the user's perceptual immersion.

Methodological Strategies: Dynamic Foveation vs. Full-Stop Mode

The core methodology evaluates two primary algorithmic strategies for peripheral animation suppression:

- **Dynamic Foveation (Adaptive Frequency Reduction):**

This strategy applies a quadratic frame-rate decay to characters located outside the central foveal cone. The animation update rate (R_i) decreases dynamically as an agent's angular distance from the focal gaze point increases, defined by the following formulation:

$$R_i = \frac{R_t}{\max\left(1, \alpha \cdot |p - p_0|^2 - af\right)}$$

While effective at maintaining motion continuity, lowering peripheral frame rates to intermediate frequencies (e.g., 5–10 Hz) frequently crosses the Critical Flicker Frequency (CFF) threshold of peripheral vision, occasionally triggering visual flicker awareness.

- **Full-Stop Mode (Complete Skeletal Freezing):**

This alternative strategy completely halts skeletal animation evaluations ($R_i = 0$) for all characters residing outside the foveal region. While their skeletal joint hierarchies are frozen in their last evaluated pose, virtual agents continue to move spatially across the environment using underlying navigation meshes (NavMesh). As soon as an agent enters the user's 5° foveal region, its skeletal graph evaluation instantly resumes. Counterintuitively, completely freezing peripheral character limb animations is significantly less perceptible to users than low-frequency updates, as it avoids crossing CFF flicker thresholds while maximizing computational savings.

Technical Implementation and Unity 3D Prototype Development

The experimental prototype was implemented within the Unity3D game engine to achieve granular runtime control over character skeletal evaluation pipelines. Rather than relying on Unity's default Animator component—which lacks fine-grained per-frame evaluation control—the framework utilizes custom evaluation pipelines driven by `PlayableGraph.Evaluate`. This API allows the rendering engine to manually pause, step forward, or resume the animation graph of each character independently without disrupting their high-level pathfinding or spatial collision logic.

Component	Specification / Framework	Technical Role in Implementation
Engine Pipeline	Unity3D Engine	Custom animation pipeline management using <code>PlayableGraph.Evaluate</code> .
Hardware Display	HTC Vive Pro Eye HMD	Immersive stereo rendering with integrated dual-camera eye tracking.
Tracking SDK	HTC SRAnipal & OpenXR	Real-time gaze point calculation and low-latency foveal raycasting.

For the virtual reality deployment, the system integrates an HTC Vive Pro Eye Head-Mounted Display (HMD) equipped with binocular eye-tracking sensors operating at

120 Hz. Real-time gaze coordinates are fetched via the HTC SRAnipal SDK through the OpenXR framework. By casting a gaze ray into the 3D scene every frame, the system dynamically calculates the 5° foveal vision cone and updates the animation evaluation state of every agent in real time with minimal latency.

Experiments Conducted with Participants

The researchers developed a virtual scene in Unity 3D with different types of crowds. The study included 12 participants, consisting of students and staff members from the institution. All participants had normal or corrected vision, and previous experience with virtual reality was not required. In addition, they used the same equipment to ensure equal testing conditions.

Two tests were conducted. In the first test, participants watched a screen and focused on a main character. They then reported when they noticed that the characters in their peripheral vision were moving more slowly or had stopped moving.

In the second test, a Vive Pro Eye virtual reality headset with eye-tracking technology was used. This device detected where each participant was looking, allowing them to freely explore the scene while the system adjusted the animations according to their gaze.

Two methods were also evaluated. Dynamic animation gradually reduced the movement update rate of characters in the peripheral area. In contrast, the Full-Stop method completely stopped the animation of characters that were not being directly observed. The purpose was to determine how much the animations could be reduced without users noticing the change.

Results Obtained

The experiments showed that it was possible to reduce the animations of characters located in the user's peripheral vision without significantly affecting the user experience. Dynamic animation reduced movement updates by 78.7%, while the Full-Stop method achieved a reduction of up to 99.3%.

Participants had greater difficulty detecting stopped characters in disorganized crowds because many characters were moving in different directions. Likewise, the small delays in eye tracking did not significantly affect the virtual reality experience.

The researchers concluded that the Full-Stop method provides the greatest resource savings because it prevents the computer from animating characters that the user is not directly observing.

Advantages, Limitations, and Future Applications

One of the main advantages is the reduction in the computer's workload, which makes it possible to display larger crowds and improve the performance of virtual reality applications. In addition, the characters being directly observed maintain their normal animations, so users hardly notice the changes.

However, the study has some limitations. Only 12 people participated, and a specific headset model was used. It may also be easier to notice a stopped character when the character is isolated, close to the user, or part of a crowd with organized movements.

This technique could be used in virtual reality video games, virtual concerts, evacuation simulations, training programs, and virtual worlds containing large numbers of characters. Future research could test it with more participants, different headsets, and a wider variety of scenarios.

Conclusión

The research demonstrated that foveated animation can reduce the computer's workload without significantly affecting the virtual reality experience. By reducing or stopping the animations of characters that the user is not directly observing, the researchers were able to save up to 99.3% of movement updates. Therefore, this technique represents an efficient solution for improving the performance of video games and virtual applications that display large crowds.

Link: <https://canva.link/fpjkp7m7ymuxfb>

Bibliographic References

Stancu, F.-V., Weiss, T., & Kuffner dos Anjos, R. (2025). Foveated animations for efficient crowd simulation. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 8(1), Article 9, 1–14.

<https://doi.org/10.1145/3728306>